Which Pixels Matter More? Entropy-Uniform Rendering for Optical Long-Context Compression

Anonymous CVPR submission

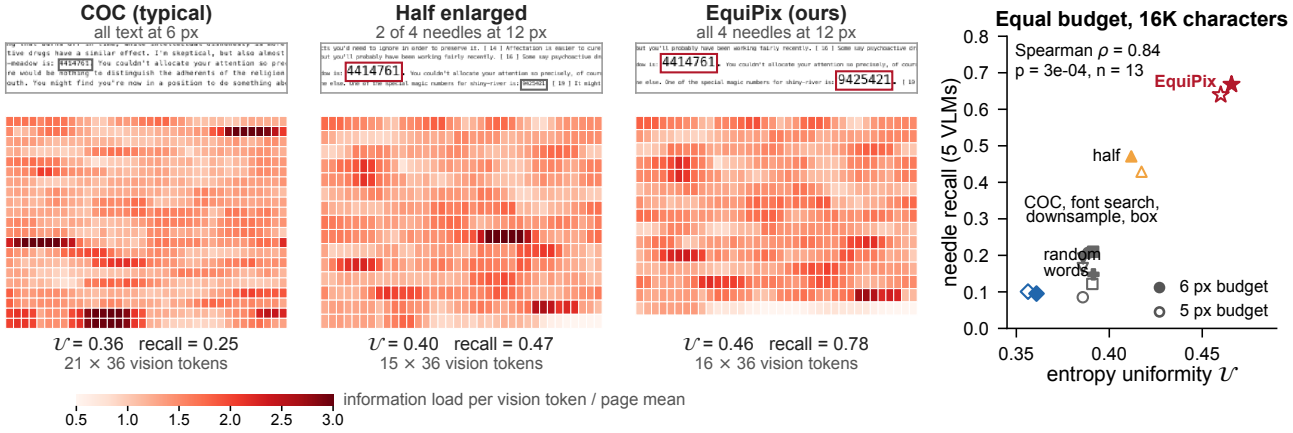


Figure 1. **Information is spread unevenly over a rendered page, and recall follows how evenly it is spread.** One 16K-character RULER document with four seven-digit needles, rendered three ways at the same vision-token budget. *Top:* crops around needles (boxed). *Bottom:* information maps; each cell is one vision token, colored by the highest load of its text relative to the page mean (Sec. 3.3), drawn at equal size although EquiPix uses fewer tokens. Under typical context optical compression (COC) the needles are hot spots; half enlargement cools two of them and EquiPix all four. Recall: five VLMs, 50 documents. *Right:* recall against entropy uniformity $\mathcal{U}$ for every rendering at the 5 and 6 px budgets.

Abstract

Context optical compression (COC) renders long text into images so that a vision-language model (VLM) can read it with several times fewer tokens. Typical COC gives every character the same pixels. We argue that this misallocates the budget, because information is spread unevenly over a rendered page. A reader can recover predictable prose from its language prior, but characters with high residual entropy, such as those of identifiers and hashes, must come from the pixels. We estimate this residual entropy from the text alone, which yields an information map of any rendered page, and we summarize the map with an entropy-uniformity score $\mathcal{U}$. Across 13 rendering–budget combinations of the same documents at an equal vision-token budget, $\mathcal{U}$ tracks needle recall over five VLMs (Spearman $\rho = 0.84$). Font search and downsampling leave $\mathcal{U}$ almost unchanged and barely help, and enlarging random words lowers both. Flattening the map is then the natural remedy. Our method, EquiPix, enlarges high-entropy strings and shrinks prose within the budget of typical COC, without training or model access. It raises RULER needle recall by 0.55 and 0.43 at the two densest budgets, and by 0.48 at 32K–128K characters with up to 27% fewer tokens per task. It also doubles the recall of flagged identifiers in LongBench documents and leaves QA accuracy unchanged. The gain shrinks at milder budgets, and every image-based method still trails plain text on QA.

1. Introduction

Long contexts are expensive to feed to large models. Context optical compression (COC) cuts this cost by rendering text into images and letting a vision-language model (VLM) read the pixels [7, 34, 41]. A page of small glyphs costs a fixed number of vision tokens however many characters it holds. In our measurements a 128K-character essay costs 27.1K prompt tokens as text and 4.7K as a typical COC image (mean of the four readers that report usage for both). COC needs no training and works with any VLM that ac-

cepts images.

Typical COC renders the whole document with one font and one size, chosen at best by a global search [7]. Every character then receives the same pixels. The reader, however, does not need the same evidence for every character. A VLM that reads rendered text combines what the glyph shows with what its language model expects. In prose the expectation fills most gaps, so blurred letters are still read correctly. In an identifier it cannot: the format of a UUID is predictable, but each hexadecimal character carries 4 bits that only the pixels can supply. Our readers show this split. Under typical COC they write the hyphens of a UUID correctly 99% of the time and its hexadecimal characters only 51% of the time, and most wrong answers keep the format and invent the content (Sec. 5.3). At an equal budget, typical COC recovers only 8–22% of RULER needles, averaged over five readers.

We ask which pixels matter more, and answer in four steps (Fig. 1). The first is a theory of where information sits after rendering. What the pixels must supply for a character is roughly its residual entropy given the context, and we estimate it from the text before anything is drawn (Sec. 3.2). The second is a measure. Residual entropy divided by glyph area gives the information load of a region in bits per vision token. Entropy uniformity $\mathcal{U}$ compares the mean load of a page with the load of its hottest regions (Sec. 3.3). $\mathcal{U}$ is 1 when every region carries the same information per pixel and small when a few regions carry far more than the rest. The third is a visualization: the load can be drawn as a heat map of any text under any rendering, which shows the informative and error-prone spans before any model is called. The fourth is the method. If uneven load causes the errors, the remedy is to flatten the map, and EquiPix does so by enlarging high-entropy strings and shrinking prose within the vision-token budget of typical COC (Sec. 3.5). It is training-free, query-independent, and needs only black-box access.

A measure is useful only if it predicts reading. Across 13 arm–budget combinations of the same documents, $\mathcal{U}$ and five-reader recall have Spearman $\rho = 0.84$. Two results do not follow from how EquiPix is built. Font search and render-then-downsample rescale every region about equally; they leave $\mathcal{U}$ almost unchanged and barely improve recall. When a random half of the needles is enlarged, the enlarged ones are recalled 74–83% of the time and the others 11–12% (Sec. 5.1).

Our contributions are:

- an information view of COC, which traces verbatim failures to characters whose residual entropy the language prior cannot supply and estimates that entropy from the text alone;
- entropy uniformity and information maps, which track recall across renderings at an equal budget and explain why rescaling the whole page (font search, downsampling) does not help;
- EquiPix, a budget-constrained glyph allocation that flattens the map. Over five readers it raises needle recall by 0.43–0.55 at 16K characters and by 0.48 at 32K–128K with up to 27% fewer tokens, doubles the recall of flagged natural identifiers, and leaves LongBench QA unchanged. The gain shrinks at milder budgets, and at 10 px font search does as well.

Verbatim anchors, which copy flagged strings into the prompt next to the image, leave the map unchanged; we treat them as optional post-processing (Sec. 5.7).

2. Related Work

Reading text from pixels has a long history, from pixel-based language models [11, 34] to unified image–text encoders [37] and screenshot pretraining [20]. Context optical compression (COC) renders long contexts to compress them: DeepSeek-OCR decodes text from few vision tokens [41, 42], Glyph searches for a good global rendering configuration [7], and other work renders whole inputs, distant context, or interleaved text [25, 39, 45]. Evaluations show where this fails: compressed reading loses retrieval ability [52], the text–pixel gap depends on font and resolution [36], optical compression can underperform mean pooling [19], and accuracy collapses when linguistic priors are removed [12, 26]. This last finding is the starting point of our account: what the prior cannot supply must come from the pixels, and our entropy maps make this quantity visible on the page. Several methods vary resolution within the input, but they are trained (FocusVTC [53], SEER [46], LensVLM [44]; in document understanding, DocOwl2 and TextMonkey [16, 28]). Among rendering changes, AGAR is the most similar: it re-renders a page and enlarges the word spans that receive the most middle-to-late-layer attention [48]. It is training-free but needs the model's attention and a first pass for each query. EquiPix is decided from the text before any model call, works with black-box APIs, and does not depend on the question.

Training-free pruning and merging reduce the tokens of an encoded image [4, 5, 10, 29, 35, 47, 49, 50], including for text-rich and OCR inputs [22, 38]. They act on visual features inside the model, where it is no longer explicit that a group of glyphs forms a hash. We do not test whether pruning keeps the enlarged glyphs. Prompt compressors drop tokens judged uninformative [17, 18, 23, 32], learned compressors map context to soft tokens [8, 13, 31], and KV-cache methods evict entries [9, 24, 43, 51]. Selective Context, like our entropy estimate, uses an information signal, but to *remove* low-information text. We keep every character and only change how large it is drawn. Caching prompt segments [14] is a competing way to cut cost for repeated queries that we do not evaluate.

We measure verbatim fidelity with RULER [15] and

collateral damage with LongBench [3]. Text-rich benchmarks [27, 30] show the same weakness on natural images and PDFs. Modern VLMs allocate vision tokens roughly in proportion to pixel area [1, 2, 6, 21, 40], so the rendering budget is a controllable cost. We report the prompt tokens counted by each provider.

3. Information in Rendered Text

We first fix the budget (Sec. 3.1), then ask which characters need pixels (Sec. 3.2), measure how evenly a rendering serves them (Sec. 3.3), and finally flatten the resulting map (Secs. 3.4 and 3.5); Fig. 2 summarizes the pipeline.

3.1. Rendering under a budget

A document $x = (c_1, \dots, c_N)$ is a sequence of characters, grouped into whitespace-separated words. A renderer R maps x and a size assignment σ (glyph height in pixels for every word) to an image $I = R(x; \sigma)$ of fixed width, which is split into pages for the VLM. Following the patch-based accounting of dynamic-resolution VLMs [40], the vision-token cost of an image with pages of width w_p and height h_p is

$$v(I) \;=\; \sum_{p} \left\lceil \tfrac{w_p}{28} \right\rceil \left\lceil \tfrac{h_p}{28} \right\rceil , \tag{1}$$

where one vision token covers 28×28 pixels (14-pixel patches merged 2×2 in Qwen2-VL). Equation (1) is used to enforce budgets; reported costs are the prompt tokens counted by each provider. Typical COC renders every character at one base size s_0, and its cost defines the budget $B = v\big(R(x; \sigma \equiv s_0)\big)$. Every method we compare stays within B, up to page rounding, and keeps every character in place.

3.2. Which pixels matter more

Consider how a reader copies a string $c_1 \dots c_L$ from the image. As a schematic model, it combines visual evidence from each glyph, which grows with the glyph size s_i, with the expectation of its language model given the context:

$$p(c_i \mid I, \mathrm{ctx}) \;\propto\; \underbrace{p(I_i \mid c_i;\, s_i)}_{\text{pixels}}\; \underbrace{p_\theta(c_i \mid \mathrm{ctx})}_{\text{language prior}} . \tag{2}$$

The pixels must supply what the prior leaves open, roughly the *residual entropy* of each character. We estimate it from the text alone, before rendering, as the surprisal under a small causal language model q,

$$h_i \;=\; -\log_2 q(c_i \mid c_{<i}), \tag{3}$$

whose expectation is the conditional entropy $H(c_i \mid c_{<i})$ under q. We use Qwen2.5-0.5B [33] and spread each token's surprisal evenly over its characters. On our RULER documents, essay prose averages 0.9 bits per character, while the hexadecimal characters of a UUID average 3.5 bits. Exact copying fails if any character fails, so a string is only as legible as its least predictable part. When the visual evidence for a high-entropy character is too weak, the prior dominates Eq. (2). It knows the format but not the content, so we expect a well-formed string with invented characters, which is what we observe (Sec. 5.3). Under a fixed budget, pixels should therefore move from low to high residual entropy. The trade is favorable because the two kinds of text sit on different parts of their legibility curves. Found English phrases keep 92–96% character accuracy from 10 down to 4 px, while a UUID at 12 instead of 5–6 px goes from 51% to 91% hexadecimal accuracy (Sec. 5.3).

3.3. Information maps and entropy uniformity

Let a_i be the area of the glyph cell of c_i in vision tokens (pixels divided by 28^2), so that $\sum_i a_i \approx v(I)$. For a window W of n consecutive characters in reading order, the *information load*

$$\ell(W) \;=\; \frac{\sum_{i \in W} h_i}{\sum_{i \in W} a_i} \tag{4}$$

is the number of bits each of its vision tokens must carry. The page mean is $\bar{\ell} = \sum_i h_i / \sum_i a_i$. Typical COC makes a_i constant, so the load follows h_i: prose windows carry about 20 bits per vision token at 6 px and windows around a UUID about 60. We define *entropy uniformity* as

$$\mathcal{U}(I) \;=\; \frac{\bar{\ell}}{\mathrm{mean}_{W \in \mathcal{T}}\, \ell(W)}, \tag{5}$$

where $\mathcal{T}$ is the set of the 160 windows with the highest load (1% of a 16K-character document; a fixed count keeps the tail from being diluted as documents grow). $\mathcal{U} = 1$ when every window carries the same load, and $\mathcal{U}$ is small when a few windows carry much more than the rest. We use $n = 48$, a little longer than a UUID with its key, so that a single rare prose word does not form a peak on its own. Other window lengths and tail statistics give similar results (Sec. A).

Three properties make $\mathcal{U}$ useful. First, it is invariant to rescaling: multiplying every a_i by a constant changes $\bar{\ell}$ and every $\ell(W)$ by the same factor. Changing the font size of the whole page or downsampling the image therefore leaves $\mathcal{U}$ almost unchanged, and only a non-uniform allocation of pixels can raise it. Second, it is computable before any model call, from the text and the layout. A low $\mathcal{U}$ under typical COC flags hot spots that reallocation could cool, although in our data it does not predict the gain of single documents (Sec. A). Third, the load can be drawn. Coloring every glyph by the load of the window centered on it, relative to $\bar{\ell}$, gives an *information map* of any page (Fig. 1, bottom). Its hot spots are the spans that the pixels

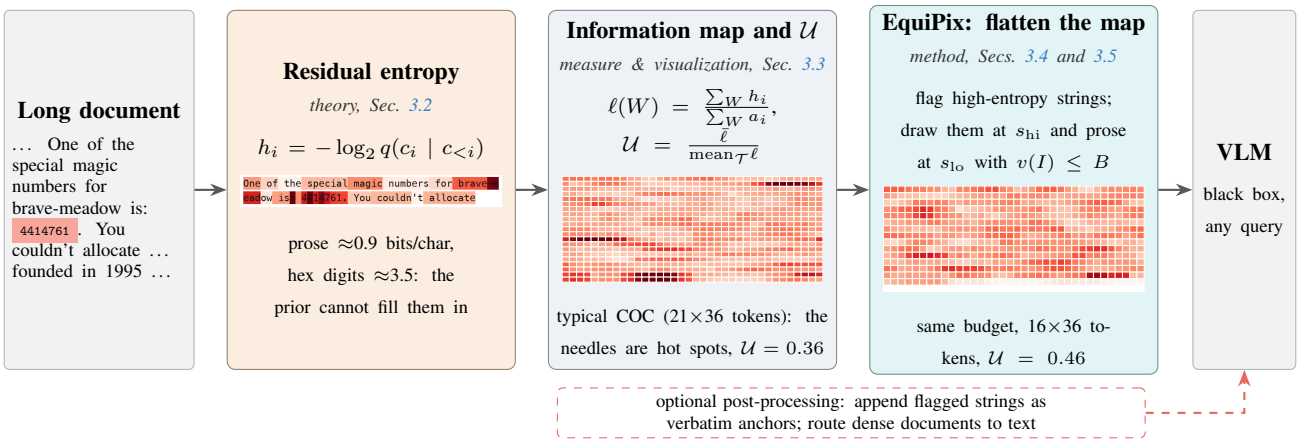


Figure 2. **From residual entropy to EquiPix.** The renderer estimates the residual entropy of every character from the text alone (left: a needle sentence colored by h_i; the key and the digits are hot). Dividing by glyph area gives the information load, which can be drawn as a map of any page and summarized by entropy uniformity $\mathcal{U}$. Typical COC leaves the high-entropy strings as hot spots. EquiPix enlarges the flagged strings and shrinks prose within the same vision-token budget, which flattens the map (same page; each cell is one vision token, as in Fig. 1). Anchors leave the map unchanged and are optional post-processing. Every step is training-free and needs no access to the model.

must carry alone, and in our data they are the spans most often misread (Sec. 5.1).

$\mathcal{U}$ is a summary of the page, not of the queried string, and it measures shape at a fixed budget. Across budgets, $\bar{\ell}$ itself changes: at 10 px every window carries less than half the load it carries at 6 px.

3.4. EquiPix: flattening the map

Ideally, glyph area would be proportional to residual entropy, $a_i \propto h_i$, which makes the load constant and $\mathcal{U} = 1$. Three constraints make this impractical: glyph sizes are integers, words must stay intact, and prose cannot shrink below the size that VLMs still read. EquiPix therefore approximates the ideal with two sizes. It flags high-entropy strings with cheap text rules instead of a language model, so that rendering stays fast and model-free, and it gives the flagged strings the pixels that prose can spare.

The detector combines regular expressions with a windowed character-entropy test. After stripping punctuation, a word is flagged if it matches one of five patterns: a hexadecimal run of at least 8 characters, a run of at least 5 digits, a path with a digit or a file extension, an IP address, or a digest prefix. Words that mix letters and digits are also flagged if they have at least 20 identifier characters or a 16-character window with at least 3.5 bits of character entropy. Four-digit numbers are excluded because in natural text they are mostly years, which are cheap to read, and long English words are never flagged. Without these exclusions the detector flagged 85–112 words per LongBench document (Sec. K). The detector selects strings that the language model also finds surprising, so EquiPix raises $\mathcal{U}$ by construction. What has to be shown empirically is that higher $\mathcal{U}$ means higher recall (Sec. 5.1).

3.5. Budget-constrained glyph allocation

Given the flagged set $\mathcal{H}(x)$, prose words are drawn at s_{lo} and flagged words at s_{hi}. Prose is wrapped at character boundaries, as in typical COC, and each flagged word is kept whole on one line. The sizes are

$$(s^*_{\text{hi}}, s^*_{\text{lo}}) = \operatorname*{lexmax}_{\substack{s_{\text{hi}} \in [s_0, 12] \\ s_{\text{lo}} \in [5, \min(s_{\text{hi}}, s_0)]}} (s_{\text{hi}}, s_{\text{lo}}) \quad \text{s.t. } v\big(R(x; \sigma_{s_{\text{lo}}, s_{\text{hi}}})\big) \le B, \tag{6}$$

where lexmax first maximizes s_{hi} and then s_{lo}, because the hottest windows limit $\mathcal{U}$. The 5 px floor keeps prose readable, and $s_{\text{lo}} \le s_0$ ensures prose is never enlarged. If no pair with $s_{\text{hi}} > s_{\text{lo}}$ fits, prose-only lines lose one pixel of line spacing; at $s_0 = 5$ px this is what makes room (Sec. I). If nothing is flagged, the image is identical to typical COC. A missed high-entropy string in a document where other strings are flagged, however, shrinks with the prose. Allocation takes about 1 s for 32K and 4 s for 128K characters on one CPU core. At $s_0 = 6$ px the result usually costs 13–27% fewer tokens than the budget, because the search stops at the largest feasible s_{hi}.

4. Experimental Setup

We treat every VLM as a black box and send one stateless request per item at temperature 0 (0.01 for MiniMax), resending unparseable outputs up to three times. The six readers span vendors, scales, and open weights: Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, GPT-6 Luna (minimal reasoning effort), Qwen3-VL-235B-A22B [1], Llama 4 Maverick,

and MiniMax-M3. Means and pooled tests use the five readers other than Claude, which was not run on every arm and is reported per reader only. Cost is the average number of prompt tokens per task as counted by each provider, including the image, the question, and any appended text. MiniMax reports no usage and is left out of cost averages only. Empty responses and transport errors (fewer than 100 of roughly 35,500 calls) are dropped; any other unparseable output counts as wrong.

Verbatim fidelity is measured with RULER needle-in-a-haystack tasks [15], re-implemented with the official templates and Paul Graham essays as the haystack. The tasks are *single* (one UUID value), *multi-key* (four keys, one queried, seven-digit values), and *multi-value* (four seven-digit values for one key). Contexts have 16K characters (Sec. 5.2), 32K–128K characters split into up to three pages (Sec. 5.4), or 32K characters at milder budgets (Sec. 5.5). Collateral damage is measured on held-out LongBench [3] MultiFieldQA-en and HotpotQA documents of at most 40K characters, drawn after the detector was frozen. We use $N = 25$ items per task and reader for RULER at 16K and 32K characters, $N = 20$ for the length study, and $N = 50$ for LongBench.

All image arms share one renderer (Menlo, 1008-pixel pages) and the budget B of typical COC at $s_0 \in \{5, 6\}$ px, about 32 and 21 characters per vision token on our long documents. Besides typical **COC**, the arms are as follows. **Font search** picks the largest uniform font and size (Menlo, Arial, Arial Narrow) that fits B, a training-free stand-in for global configuration search [7]. **Downsample** renders at 10 px and shrinks the image to B [41]. **Random words** applies our allocation to random ordinary words whose total length matches the flagged strings. **Box** uses our layout without enlargement and draws a box around each flagged string. **Half** applies our allocation to a random half of the flagged strings. As post-processing, verbatim anchors (the flagged strings copied into the prompt) are appended to COC and to EquiPix (**COC+anchor**, **EquiPix +anchor**), and **text** gives the plain text.

RULER is scored by recall of the gold values (case-insensitive containment) and LongBench by token-level F1, as in the original benchmarks. Because recall does not penalize extra answers, we also report a precision-aware F1. For each reader, task, and setting (a *cell*) we compute the mean paired difference over items with a 95% bootstrap interval. Pooled effects average over cells, with intervals from a bootstrap over readers and items, and significant cells are counted after Holm correction (Sec. D).

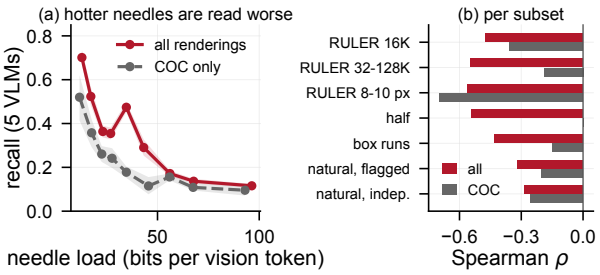


Figure 3. **The hot spots of the map are where reading fails.** (a) Recall of each needle against the information load of the 48-character window around it, binned (95% CI), over all renderings and settings (3,760 needle–image pairs) and over COC images only (1,030). (b) Spearman ρ between needle load and recall per subset.

5. Results

5.1. Does entropy uniformity predict recall?

Figure 1 (right) plots five-reader recall against $\mathcal{U}$ for every rendering of the 16K-character RULER documents at the 5 and 6 px budgets, 13 arm–budget points in all. The two are rank-correlated with Spearman $\rho = 0.84$ ($p = 3 \times 10^{-4}$). Random words have the lowest uniformity (0.36) and recall (0.10). COC, font search, downsampling, and the box control sit together at $\mathcal{U} = 0.385$–0.391 with recall 0.09–0.21; half enlargement reaches 0.41–0.42 and 0.43–0.47, and EquiPix 0.46–0.47 and 0.64–0.67. Over single documents the correlation is weaker ($\rho = 0.48$ over 1,075 images), because $\mathcal{U}$ summarizes a page while each question asks about one string.

Because the detector targets high-entropy strings, EquiPix raises $\mathcal{U}$ by construction, so the correlation would be circular if it rested on EquiPix alone. Three groups of points do not depend on the detector's choices. Font search and downsampling change the size of every glyph, which leaves $\mathcal{U}$ almost unchanged (Sec. 3.3). As predicted, they barely change recall (+0.04 and +0.08 at 5 px, −0.01 and −0.03 at 6 px). Random words receive the same allocation as EquiPix, but on low-entropy text. The needles shrink with the prose, the hot spots get hotter, and recall falls below COC at 6 px. Half enlargement picks the enlarged strings at random, so its two halves form a controlled comparison within the same images. Enlarged needles are recalled 74% (5 px) and 83% (6 px) of the time, the others 12% and 11%. At 6 px the non-enlarged needles do worse than under COC (0.24), because they shrink to 5 px with the prose and their load rises. Over the whole document, half enlargement lands between COC and EquiPix (+0.21 to +0.30 over COC, and 0.31 below EquiPix; Tab. 2).

The map also predicts which strings fail, without reference to any arm. For every needle we take the load of

Table 1. **Equal-budget comparison on RULER needle retrieval (16K characters).** Recall per reader, averaged over three NIAH tasks (N=25 each). All image columns share the vision-token budget of typical COC at 5 or 6 px. **Bold**: best within each group. Mean rows exclude Claude‡, which was not run on random words or EquiPix +anchor. Pooled over tasks and readers, EquiPix −COC is +0.55 [0.48, 0.62] at 5 px and +0.43 [0.27, 0.56] at 6 px (95% CI; Tab. 2). On these tasks the anchors contain the answer, so the anchor columns measure copying rather than reading (Sec. 5.7).

	5 px budget (≈32 chars per vision token)							6 px budget (≈21 chars per vision token)						
	equal-budget image					+ anchors (post-proc.)		equal-budget image					+ anchors (post-proc.)	
Reader	COC one size	Font search	Down-sample	Random words	EquiPix (ours)	COC +anchor	EquiPix +anchor	COC one size	Font search	Down-sample	Random words	EquiPix (ours)	COC +anchor	EquiPix +anchor
Claude Sonnet 5.5‡	0.14	0.22	0.19	–	**0.79**	**0.95**	–	0.57	0.64	0.46	–	**0.92**	**1.00**	–
Gemini 3.5 Flash-Lite	0.41	0.57	0.79	0.48	**1.00**	**1.00**	**1.00**	0.88	0.78	0.84	0.46	**1.00**	1.00	**1.00**
GPT-6 Luna	0.02	0.03	0.02	0.02	**0.50**	0.91	**0.94**	0.11	0.16	0.08	0.02	**0.65**	0.82	**0.98**
Qwen3-VL-235B	0.00	0.00	0.00	0.00	**0.55**	**1.00**	**1.00**	0.07	0.08	0.03	0.00	**0.60**	**1.00**	**1.00**
Llama 4 Maverick	0.00	0.00	0.00	0.00	**0.58**	**0.97**	**0.97**	0.00	0.01	0.00	0.00	**0.59**	0.90	**1.00**
MiniMax-M3	0.00	0.00	0.00	0.00	**0.57**	**1.00**	0.98	0.02	0.01	0.00	0.00	**0.42**	0.97	**1.00**
Mean recall	0.08	0.12	0.16	0.10	**0.64**	0.97	**0.98**	0.22	0.21	0.19	0.10	**0.65**	0.94	**1.00**
Mean F1	0.09	0.12	0.16	0.10	**0.55**	0.80	**0.80**	0.23	0.21	0.19	0.10	**0.56**	0.79	**0.83**
Avg. tokens / task	815	811	804	814	818	860	857	920	902	894	844	842	968	881

the window around it and the share of readers that recalled it (Fig. 3). Over all renderings, recall falls from 0.70 below 15 bits per vision token to 0.12 above 80 (Spearman $\rho = -0.50$, 3,760 needle–image pairs). The relation holds within COC images alone, where nothing was reallocated ($\rho = -0.36$ over 1,030 needles; 0.52 to 0.10 across the same range), and in every subset we ran. It is weakest on the natural strings ($\rho = -0.20$ and -0.25 under COC, 60 targets each, $p = 0.12$ and 0.05), whose targets the detector did not always choose.

$\mathcal{U}$ measures the shape of the load at a given budget, not its level. In every other setting (8 and 10 px, 32K–128K characters), EquiPix has both the higher $\mathcal{U}$ and the higher recall. At 8 and 10 px, where font search was also run, $\mathcal{U}$ and recall rank the three arms identically. Across settings the mean load $\bar{\ell}$ also changes, yet pooling all 25 points still gives $\rho = 0.80$. With other window lengths and tail statistics, ρ at 16K ranges from 0.78 to 0.90 (Sec. A).

5.2. Equal-budget comparison

Table 1 compares all arms at the same vision-token budget. Averaged over five readers, typical COC recovers 8% of needles at the denser budget and 22% at the lighter one; only Claude (0.57) and Gemini (0.88) exceed 0.11 at 6 px. EquiPix raises the mean to 0.64 and 0.65, a pooled gain of 0.55 and 0.43 (Tab. 2), significant in 14 and 11 of 15 cells after Holm correction and never significantly negative. Font search reaches 0.12 and 0.21, and downsampling 0.16 and 0.19. At 6 px EquiPix also uses 8% fewer tokens per task than COC (842 vs. 920), because prose shrinks more than the enlarged strings grow. The weaker readers gain most (Llama 4 Maverick from 0.00 to 0.59, GPT from 0.11 to 0.65 at 6 px), and Gemini, already at 0.88, gains 0.12. Under the precision-aware F1 the gains are smaller (0.46 and 0.34 pooled) but remain significant in at least 9 of 15 cells. Two controls isolate the ingredients. Random words, with sizes and cost close to those of EquiPix, recover 0.10 of the needles at both budgets, 0.54 and 0.56 below EquiPix. The box control makes the needle easy to find but not easier to read. It is 0.53 below EquiPix and slightly below COC (−0.05, interval upper bound −0.001), perhaps because the

Table 2. **Pooled effects** over the five non-Claude readers (recall unless noted). CI: 95% bootstrap over readers and items. Sig.: cells (reader × task) significantly positive after Holm correction within each row; neg.: significantly negative.

Comparison	Δ	95% CI	sig.	neg.
Equal budget, RULER				
EquiPix − COC, 16K, 5 px	+0.55	[+0.48, +0.62]	14/15	0
EquiPix − COC, 16K, 6 px	+0.43	[+0.27, +0.56]	11/15	0
EquiPix − COC, 32K–128K, 6 px	+0.48	[+0.43, +0.53]	35/45	0
EquiPix − font search, 6 px	+0.44	[+0.31, +0.55]	11/15	0
EquiPix − random words, 6 px	+0.56	[+0.47, +0.64]	12/15	0
box − COC, 6 px	-0.05	[-0.13, 0.00]	0/15	1
EquiPix − box, 6 px	+0.53	[+0.41, +0.64]	14/15	0
half − COC, 5 px	+0.30	[+0.21, +0.40]	9/10	0
half − COC, 6 px	+0.21	[0.00, +0.39]	8/10	0
EquiPix − half, 6 px	+0.31	[+0.20, +0.41]	6/10	0
Milder budgets (32K)				
EquiPix − COC, 10 px	+0.20	[+0.11, +0.30]	4/10	0
EquiPix − font search, 10 px	+0.04	[-0.04, +0.12]	1/10	1
Natural strings (LongBench documents)				
EquiPix − COC, flagged targets	+0.25	[+0.12, +0.38]	4/5	0
EquiPix − COC, detector-indep.	-0.01	[-0.11, +0.08]	0/5	0
Ordinary comprehension (LongBench F1)				
EquiPix − COC	+0.02	[-0.02, +0.06]	0/10	0
EquiPix +anchor − text	-0.21	[-0.29, -0.13]	0/10	9
Post-processing: EquiPix+anchor − COC+anchor (all settings: Tabs. 8 and 9)				
RULER 16K, 6 px (F1)	+0.04	[0.00, +0.08]	1/15	0
multi-key 64K–128K (F1)	-0.01	[-0.05, +0.03]	0/10	0
natural, flagged, 6 px	+0.02	[-0.06, +0.10]	0/5	0

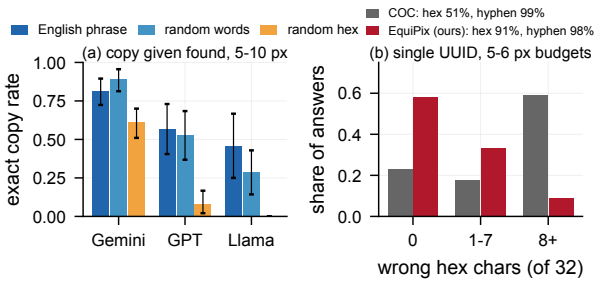


Figure 4. **Reading is limited by what the language prior cannot supply.** (a) 32-character targets of three types hidden in the same 16K-character documents and rendered by COC at 5–10 px: share copied exactly among answers that found the target (95% CI), for the three readers that found enough targets of every type. (b) Single-UUID needles at the 5 and 6 px budgets: wrong hexadecimal characters per answer of the right length. Under COC most such answers are fabricated; under EquiPix most are exact.

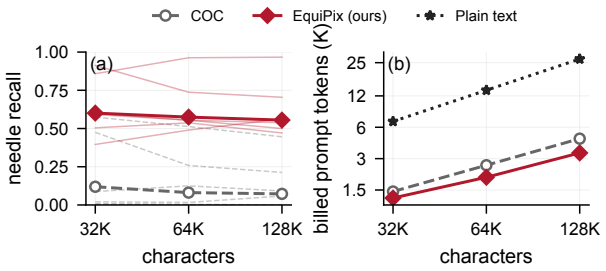


Figure 5. **Context length.** RULER at 32K–128K characters (6 px budget, up to three pages, N=20 per task). (a) Needle recall; thin lines are individual readers (including Claude), thick lines the mean of five readers. (b) Average prompt tokens per task over the four of these readers that report usage; log scale.

box line touches the small glyphs. The gain thus comes from making the high-entropy strings legible, not from layout changes or from pointing at them.

5.3. Why does flattening help?

The account of Sec. 3.2 predicts that readers fail where the pixels must supply what the prior cannot. The single-UUID needles of Tab. 1 agree. Under COC a third of the answers are empty. In answers of the right length the four hyphens, which the format predicts, are correct 99% of the time and the 32 hexadecimal characters 51%. EquiPix raises the hexadecimal characters to 91% and leaves the hyphens at 98%, and the gain holds for every reader (from 0.15–0.96 to 0.82–1.00). Wrong answers are mostly fabricated rather than misread. Of the right-length answers, 59% have at least 8 of 32 characters wrong (median 27, close to the 30 expected from guessing), and only 23% are exact (Fig. 4b). The weaker readers drive this pattern; Gemini almost never fabricates and errs by misreading (Sec. G). Under EquiPix, 58% of the answers are exact and 9% fabricated. Among partial misreads under COC, the most frequent confusion is 8→0, two glyphs that differ by a small stroke.

To vary predictability at fixed glyph size, we hid one 32-character value in the same documents and changed only its type: an English phrase, random common words, or random hexadecimal characters, rendered by COC at 4–10 px. Three readers found enough targets of every type. For each, a found hexadecimal string is copied exactly far less often than a found English phrase: 0.61 vs. 0.82 for Gemini, 0.08 vs. 0.57 for GPT, and 0.00 vs. 0.46 for Llama (Fig. 4a). Hexadecimal and English targets also differ in their glyphs, so this test supports but does not isolate the role of the prior (Sec. G).

5.4. Scaling with context length

COC is most attractive for long documents, so Fig. 5 extends the comparison to 128K characters. Under COC, mean recall declines from 0.12 at 32K to 0.08 at 64K and 0.07 at 128K, consistent with reports that compressed reading loses retrieval ability as context grows [52]. EquiPix stays between 0.60 and 0.56, a pooled gain of 0.48 [0.43, 0.53] with 35 of 45 cells significant after Holm correction. It uses 13%, 23%, and 27% fewer tokens per task than COC at the three lengths, and 5.4×, 6.8×, and 8.0× fewer than plain text, which every reader answers with recall of at least 0.99. Readers differ widely (0.47 to 0.97 at 128K for EquiPix; Tab. 5).

5.5. Milder budgets

Our main budgets put most readers near the floor of COC, where gains are easiest. We therefore repeated single and multi-key at 32K characters with base sizes of 8 and 10 px, which cost 1.5 and 2.3 times as many vision tokens as 6 px. COC improves with size (multi-key 0.26 at 8 px and 0.60 at 10 px), and the gain of EquiPix shrinks to 0.38 at 8 px and 0.20 at 10 px. Font search catches up: EquiPix beats it by 0.26 at 8 px but only by 0.04 at 10 px, which is not significant. Single UUIDs remain hard (COC 0.20, EquiPix 0.44 at 10 px). All pooled comparisons at 5, 6, 7, and 8 px, with recall and F1, are listed in Tabs. 8 and 9 of the supplementary, and the natural identifiers at 5–7 px in Tab. 18. Reallocation helps most where the budget is tight, which is also where COC saves the most.

5.6. Natural strings and ordinary comprehension

RULER needles are synthetic, and the detector was designed with such strings in mind. We therefore built two sets of 60 questions from LongBench documents of up to 40K characters, each asking for the exact string that follows a unique six-word phrase (Sec. M). For targets the detec-

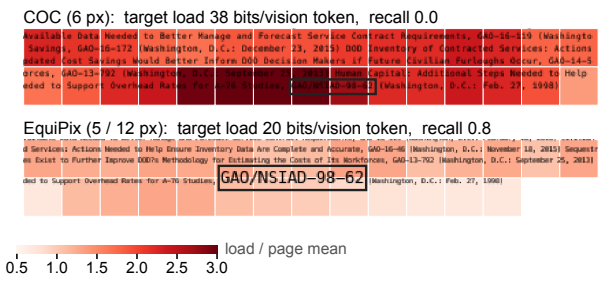


Figure 6. **Information map of a LongBench document** (GovReport, 30K characters) around the target `GAO/NSIAD-98-62` (boxed), in the style of Fig. 1: each cell is one vision token, colored by load relative to the page mean. Under COC the target sits in the hottest cells and no reader copies it. EquiPix halves its load and four of five readers copy it, while the unflagged citations around it stay hot.

tor flags, such as report and grant numbers, ISBNs, citation keys, and URLs, EquiPix doubles mean recall (0.23 to 0.48, significant for four of five readers) at 12% fewer tokens per task (Tab. 17). For digit-bearing targets sampled without the detector, mostly short numbers and version strings such as "25,000" or "2.0.13", only one of 60 is flagged. Every image arm is poor on these (0.36–0.45 against 0.96 for text), and EquiPix neither helps nor hurts (-0.01 [-0.11, 0.08]). A synthetic ablation agrees: needles the regex misses leave EquiPix at COC recall (0.33 vs. 0.32), whereas an oracle detector would reach 0.82 (Tab. 7).

The maps explain both results at the level of single strings. On flagged targets EquiPix halves the mean load (27.8 to 14.2 bits per vision token), while on detector-independent targets the load rises from 20.5 to 23.0 because they shrink with the prose (Fig. 6). Page-level $\mathcal{U}$, however, does not rise on these documents (0.374 to 0.365 on the flagged set). Natural documents contain many unflagged hot spots, such as names, short numbers, and citations, which grow hotter when prose shrinks. The regex detector is enough to flatten RULER pages but cools only the flagged spots of natural ones; choosing what to enlarge from the map itself is the natural next step. Shrinking prose could also hurt questions unrelated to identifiers. On held-out LongBench QA (Tab. 6), F1 changes by +0.02 [-0.02, 0.06] relative to COC, which rules out losses larger than about 0.02; one reader–task cell is significantly negative. Every image arm trails plain text: EquiPix +anchor loses 0.15–0.33 F1 in 9 of 10 cells, at a fifth to a quarter of the text cost. EquiPix removes a failure specific to identifiers but does not make COC a substitute for text.

5.7. Anchors as post-processing

Anchors append the flagged strings verbatim after the question. They leave the map unchanged, so we treat them as post-processing that can be added to either image. In the RULER essays the only flagged strings are the needles, so the anchor list is the answer and both anchor arms reach 0.90–1.00 by copying (Tab. 1). On flagged natural identifiers, where documents carry 5.3 anchors on average, EquiPix +anchor lies on or above the recall–cost frontier of COC+anchor by 0.02–0.05 across 5–7 px, though no pooled interval excludes zero. At 64K–128K characters it costs 29–31% less (Sec. H). When the target is not flagged, anchors can mislead (0.36 vs. 0.44 without them). Beyond about 16 flagged strings per 32K characters, anchors on either image become expensive and routing to text takes over (Sec. L).

6. Limitations and Conclusion

Our residual-entropy estimate is the surprisal of a small left-to-right language model. It ignores the right-hand context and the visual distinctiveness of glyphs, which is why we smooth it over windows and use $\mathcal{U}$ to compare renderings of the same documents rather than as an absolute score. The correlation with recall rests on 13 arm–budget points from synthetic needles, and EquiPix raises $\mathcal{U}$ partly by construction. On natural documents the regex detector cools the flagged targets but leaves page-level $\mathcal{U}$ unchanged. The gains are largest at dense budgets where COC fails; at 10 px font search does as well. COC, with or without reallocation, trails plain text on ordinary QA by up to 0.33 F1. Our natural sets are small (60 automatically constructed questions each), the detector protects only high-entropy strings, and anchors can mislead when the target is not flagged. We test black-box readers through public APIs whose versions and preprocessing may change, and we do not test trained optical compressors [7, 41] or token pruning. Budgets follow one patch-based rule (Eq. (1)), and token savings have exceptions at short lengths (Gemini up to 32K characters, Qwen at 16K and 5 px, and Llama, whose usage ignores image size). We use ASCII English only and do not measure latency, prompt caching [14], or whether text inside images bypasses text-side safety filters.

A rendered page does not carry information evenly. The renderer can see this before any model call: residual entropy, estimated from the text, gives an information map of the page, and entropy uniformity summarizes it. At an equal budget, renderings with a flatter map are read better, and renderings that only rescale the page leave both unchanged. EquiPix flattens the map by moving pixels from prose to high-entropy strings. Across five readers it turns fabricated identifiers into read ones, raises needle recall by about half the scale at dense budgets, and doubles the recall of flagged natural identifiers. The gap to plain text on ordinary QA remains.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. 3, 4

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. 3

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024. 3, 5

[4] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *Proceedings of the International Conference on Learning Representations*, 2023. 2

[5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *Proceedings of the European Conference on Computer Vision*, pages 19–35, 2024. 2

[6] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024. 3

[7] Jiale Cheng, Yusen Liu, Xinyu Zhang, Yulin Fei, Wenyi Hong, Ruiliang Lyu, Weihan Wang, Zhe Su, Xiaotao Gu, Xiao Liu, et al. Glyph: Scaling context windows via visual-text compression. *arXiv preprint arXiv:2510.17800*, 2025. 1, 2, 5, 8

[8] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023. 2

[9] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025. 2

[10] Zhengyao Fang, Pengyuan Lyu, Chengquan Zhang, Guangming Lu, Jun Yu, and Wenjie Pei. Prune redundancy, preserve essence: Vision token compression in VLMs via synergistic importance-diversity. In *Proceedings of the International Conference on Learning Representations*, 2026. 2

[11] Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024. 2

[12] Yonghan Gao, Zehong Chen, Lijian Xu, Jingzhi Chen, Jingwei Guan, and Xingyu Zeng. ZeroSense: How vision matters in long context compression. *arXiv preprint arXiv:2603.11846*, 2026. 2

[13] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *Proceedings of the International Conference on Learning Representations*, 2024. 2

[14] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. 2, 8

[15] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Proceedings of the First Conference on Language Modeling*, 2024. 2, 5

[16] Anwen Hu, Haiyang Xu, Liang Zhang, Jiabo Ye, Ming Yan, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mPLUG-DocOwl2: High-resolution compressing for OCR-free multi-page document understanding. *arXiv preprint arXiv:2409.03420*, 2024. 2

[17] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. 2

[18] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. 2

[19] Ivan Yee Lee, Cheng Yang, and Taylor Berg-Kirkpatrick. Optical context compression is just (bad) autoencoding. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[20] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18893–18912, 2023. 2

[21] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025. 3

[22] Jianjian Li, Junquan Fan, Feng Tang, Gang Huang, Shitao Zhu, Songlin Liu, Nian Xie, Wulong Liu, and Yong Liao. FCoT-VL: Advancing text-oriented large vision-language models with efficient visual token compression. *arXiv preprint arXiv:2502.18512*, 2025. 2

[23] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023. 2

[24] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural*

Information Processing Systems, pages 22947–22970, 2024. 2

[25] Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10564–10578, 2025. 2

[26] Yunhao Liang, Ruixuan Ying, Bo Li, Hong Li, Kai Yan, Qingwen Li, Min Yang, Satoshi Okamoto, Zhe Cui, and Shiwen Ni. Visual merit or linguistic crutch? a close look at DeepSeek-OCR. *arXiv preprint arXiv:2601.03714*, 2026. 2

[27] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024. 3

[28] Yuliang Liu, Biao Yang, Qiang Liu, Zhang Li, Zhiyin Ma, Shuo Zhang, and Xiang Bai. TextMonkey: An OCR-free large multimodal model for understanding document. *arXiv preprint arXiv:2403.04473*, 2024. 2

[29] Guangchuan Lv, Dianxing Shi, and Dingjie Fu. VPRune: Efficient training-free pre-LLM visual token pruning. *arXiv preprint arXiv:2609.24485*, 2026. 2

[30] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, et al. MMLongBench-Doc: Benchmarking long-context document understanding with visualizations. In *Advances in Neural Information Processing Systems*, pages 95963–96010, 2024. 3

[31] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, pages 19327–19352, 2023. 2

[32] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. 2

[33] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. 3

[34] Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *Proceedings of the International Conference on Learning Representations*, 2023. 1, 2

[35] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22857–22867, 2025. 2

[36] Kaiser Sun, Xiaochuang Yuan, Hongjun Liu, Chen Zhao, Cheng Zhang, Mark Dredze, and Fan Bai. Reading, not thinking: Understanding and bridging the modality gap when text becomes pixels in multimodal LLMs. *arXiv preprint arXiv:2603.09095*, 2026. 2

[37] Michael Tschannen, Basil Mustafa, and Neil Houlsby. CLIPPO: Image-and-language understanding from pixels only. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11006–11017, 2023. 2

[38] Ben Wan, Yan Feng, Zihan Tang, Weizhe Huang, Yuting Zeng, Jia Wang, and Tongxuan Liu. RTPrune: Reading-twice inspired token pruning for efficient DeepSeek-OCR inference. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 124471–124491, 2026. 2

[39] Alex Jinpeng Wang, Linjie Li, Yiqi Lin, Min Li, Lijuan Wang, and Mike Zheng Shou. Leveraging visual tokens for extended text contexts in multi-modal learning. In *Advances in Neural Information Processing Systems*, pages 14325–14348, 2024. 2

[40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3

[41] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025. 1, 2, 5, 8

[42] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026. 2

[43] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *Proceedings of the International Conference on Learning Representations*, 2024. 2

[44] Roy Xie, Dan Friedman, Donghan Yu, Bowen Pan, Christopher Fifty, Jang-Hyun Kim, Xianzhi Du, Zhe Gan, Vivek Rathod, and Bhuwan Dhingra. LensVLM: Selective context expansion for compressed visual representation of text. *arXiv preprint arXiv:2605.07019*, 2026. 2

[45] Ling Xing, Alex Jinpeng Wang, Rui Yan, Xiangbo Shu, and Jinhui Tang. Vision-centric token compression in large language model. In *Advances in Neural Information Processing Systems*, pages 33080–33110, 2025. 2

[46] Jiawei Xu, Zhilin Zhai, Jinrui Fang, Ruohan Xu, Mingfei Lu, Yi Zhang, Guanchu Wang, Tianlong Chen, and Ying Ding. SEER: Long-context reasoning via selective visual-text compression. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[47] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19792–19802, 2025. 2

[48] Shenglai Zeng, Qirui Wang, Kai Guo, Xinnan Dai, Xianxuan Long, and Hui Liu. Magnifying what matters: Attention-guided adaptive rendering for visual text comprehension. *arXiv preprint arXiv:2606.12898*, 2026. 2

[49] Haokui Zhang, Congyang Ou, Dawei Yan, Peng Wang, Qingsen Yan, Yu Zhang, Ying Li, and Rong Xiao. TRIO: Token reduction via inference-objective guidance for efficient vision-language models. *arXiv preprint arXiv:2602.04657*, 2026. 2

[50] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A. Gudovskiy, Tomoyuki

Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 74840–74857, 2025. 2

[51] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, pages 34661–34710, 2023. 2

[52] Hongbo Zhao, Meng Wang, Fei Zhu, Wenzhuo Liu, Bolin Ni, Fanhu Zeng, Gaofeng Meng, and Zhaoxiang Zhang. VTCBench: Can vision-language models understand long context with vision-text compression? *arXiv preprint arXiv:2512.15649*, 2025. 2, 7

[53] FangZhi Zhong, Xuerui Qiu, Yuqi Pan, Ya Liu, Shaowei Gu, Bo Xu, and Guoqi Li. FocusVTC: Efficient and high-performance visual text compression with adaptive resolution. *arXiv preprint arXiv:2609.36651*, 2026. 2

LLM Usage Statement

Large language models are the *objects of study* in this work: the six vision-language models listed in Sec. 4 and the additional Claude reader of Sec. O were queried as black-box readers, and all reported numbers are computed by our scripts from their logged outputs. Every number in the paper is produced programmatically from the experiment logs, and every reference was checked against its primary source. No LLM was used to generate or label evaluation data beyond the readers' answers being evaluated.